

NUFC 팬 투표 플랫폼 · 성능 개선

개발자의 밥줄을 찾아서

성능을 고치러 갔더니 고칠 게 없었고, 대신 틀린 값이 나왔다

① Next.js 렌더링 · 캐싱 정책

② SQL 최적화

LCP 14.4s → 2.95s

① 렌더링·캐싱 + hydration 수정 결과
Performance 73 → 89

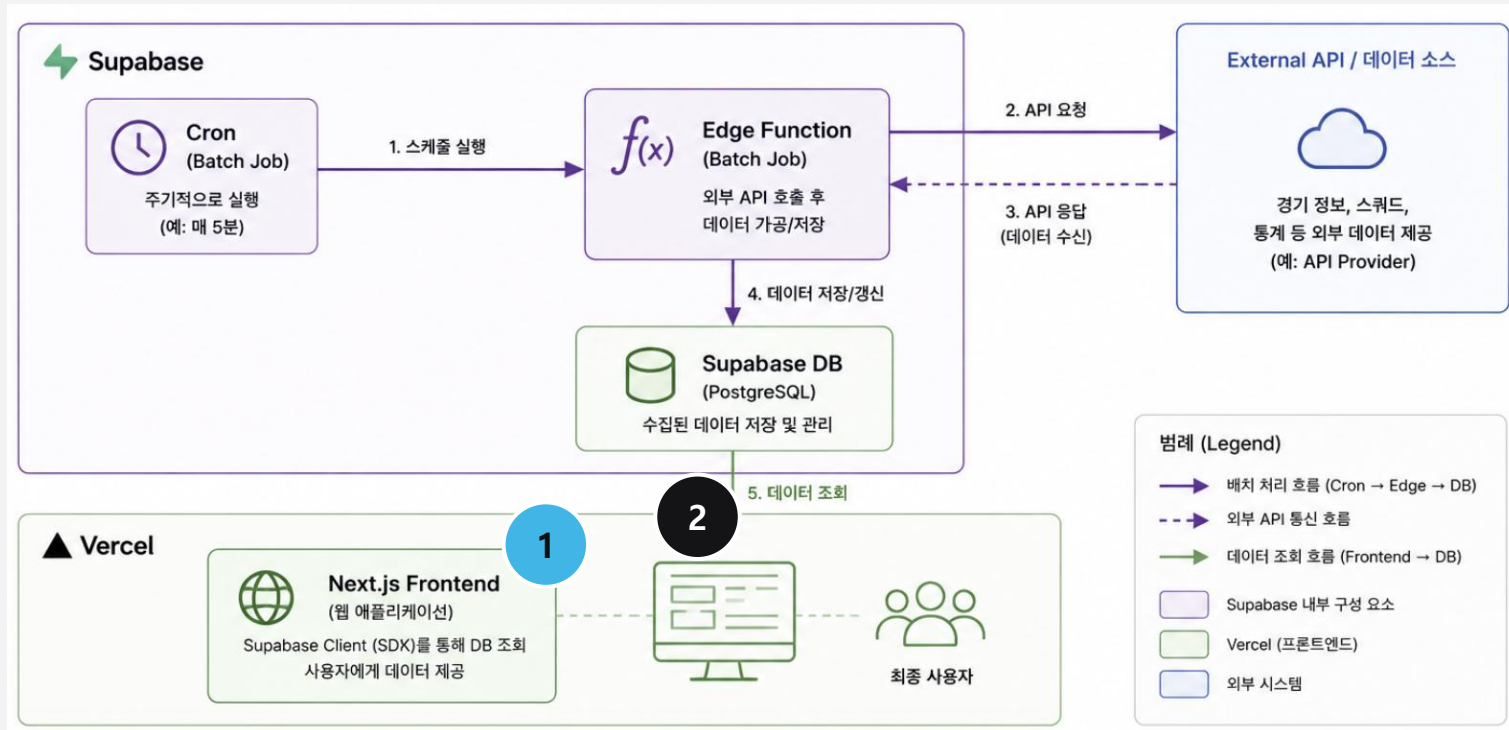
참여자 500 → 72

② 표시돼야 할 500이 72로 보였다

2026-08-26 · 발표 5분

인프라 분석 — 개선 가능한 지점 두 곳

먼저 인프라부터 뜯어봤다. 고칠 수 있는 곳이 두 군데 나왔다.



이 두 번호가 발표 내내 이정표입니다.

1 데이터를 언제 가져오는가

Next.js 렌더링 · 캐싱 정책

- 조회가 브라우저 useEffect에서 출발한다
- 라우트·쿼리 단위 캐싱 정책이 없다

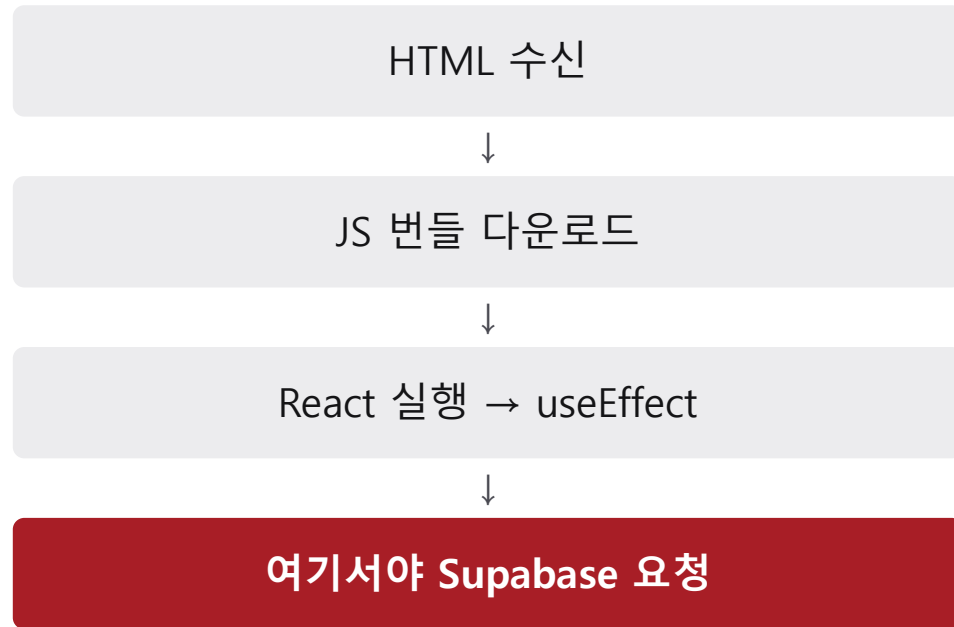
2 데이터를 얼마나 가져오는가

SQL 최적화

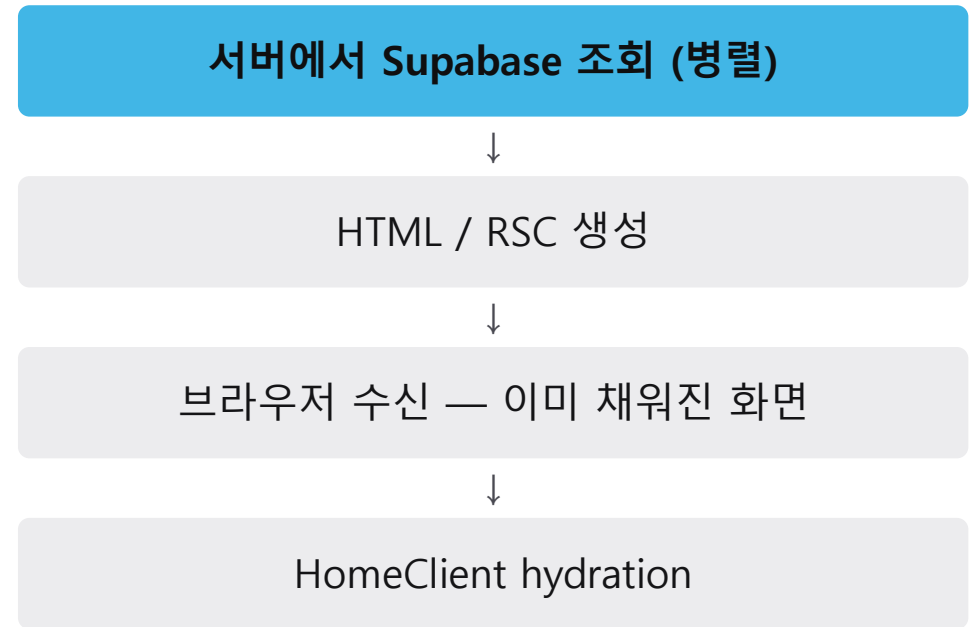
- 상한 없는 select가 28곳 중 23곳
- 참여자 집계를 DB가 아니라 JS에서 한다

① 문제는 쿼리가 아니라 출발선

Before — 데이터 요청이 세 걸음 늦게 출발



After — 데이터가 HTML보다 먼저



번들 다운로드는 HTML 파싱 중 시작되지만 데이터 요청은 직렬이다. 요청 시작점을 «브라우저 JS 실행 이후»에서 «서버가 HTML을 만들기 전»으로 옮겼다. 서로 의존하지 않는 조회는 Promise.all로 병렬 발사.

① 캐싱은 별개 작업이 아니라 짝

조회를 서버로 옮기면 그 시간이 TTFB로 청구된다. TTL 정책이 그 청구서를 막는다.

데이터 성격에 따라 TTL을 나눴다

홈 투표 섹션 / 투표 목록

참여 수가 계속 변한다

30초

선수 후보 명단

사실상 시즌 단위 고정

3600초

내 예측 제출 내역

사용자별 데이터

캐시 제외

LCP

14.4s → 4.89s → 2.95s

① 렌더링·캐싱 → hydration 수정 · 약 -80%

서버 응답 시간

21ms

«서버로 옮기면 TTFB가 늦어진다»에 대한 답

정책은 눈에 보이지 않아 리팩터링 중 조용히 지워진다. cache-policy.test.mjs / prefetch-policy.test.mjs로 고정해 뒀다.

① → ②

그럼 그 캐시에 담긴 값은 맞는가?

`/polls` 는 당시 빌드 산출물에서 `o (Static)`

빌드 시점에 만든 HTML을 계속 내보낸다 — `revalidate` 선언이 없어 잘린 집계가 그대로 굳는다.

① 이 만든 정적 생성·캐싱 구조는 맞는 값만 굳히는 게 아니다. 틀린 값도 똑같이 굳힌다.

① → ②

② 트래픽이 0인 서비스에서 뭘 재나

52행짜리 테이블은 무엇을 어떻게 짜도 빠르다. 임의의 10만 표 대신, 서비스 목표에서 규모를 유도했다.

$$\text{DAU 100명} \times \text{운영 중인 투표 10개} \times \text{1인 1표} = \text{1,000표}$$

선택형

votes

제출 1회당

1행

평점형

rating_votes

제출 1회당 · 선수 수만큼

14행

같은 «사용자 100명»인데 한쪽은 500행, 다른 쪽은 7,000행이다. 사용자 수만 보고 규모를 가늠하면 14배를 놓친다.

② 예상과 실측

목표 규모를 데이터로 만들어 부하를 걸었다. 두 칸 다 예상이 틀렸다.

성능

예상 — 병목을 찾을 것

77배

여유 · 고칠 것이 없다

현실 피크 0.33 세션/s vs 안정 처리 25.47 세션/s

정확성

예상 — 문제 없음

500 → 72

화면에 표시되는 참여자 수

에러 없음 · HTTP 200 · 정상 배열

원인 Supabase의 db-max-rows = 1,000 — 상한 없는 select가 조용히 잘린다 (PostgreSQL 자체 기본값은 무제한)

임계값 선수 14명 기준, 목록에 평점 투표 5개면 참여자 14명에서 이미 틀린다

가정 $500 = 100\text{명} \times \text{평점 투표 5개}$ · 현실 피크 0.33 세션/s = 100명 / 300초 전원 유입

수정 `count(distinct user_id)` 뷰로 전환 → 같은 시드에서 5개 전부 100 (구방식 1개 72 · 4개 누락)

배운 것

- 1 성능 문제는 «무엇이 느린가»보다
«어디서 시작하는가»를 먼저 본다

같은 쿼리인데 시작 시점만 옮겨서 LCP가 1/3이 됐다

- 2 최적화는 트레이드오프다.
캐싱은 그 짝이지 별개 작업이 아니다

서버로 옮긴 조회 시간은 TTFB로 청구된다

달한 과제 ✓ revalidate = 30 ✓ hydration mismatch ✓ Cache-Control ✓ 참여자 집계 뷰

두 지점 다 부하를 걸어보기 전에는 몰랐다. 측정하지 않으면 개선했다고도, 멀쩡하다고도 말할 수 없다.